



■ ■: Vector Databases and RAG

■ ■ ■: ■ ■ ■
■ ■: 2025 ■ 3 ■ 15 ■
■ ■: CS301

1. Introduction to Vector Databases

Vector databases are specialized database systems designed to store and query high-dimensional vectors efficiently. Unlike traditional databases that store structured data in rows and columns, vector databases excel at handling unstructured data like text, images, and audio by converting them into numerical representations called embeddings. These databases use advanced indexing techniques such as HNSW (Hierarchical Navigable Small World) graphs and IVF (Inverted File Index) to enable fast approximate nearest neighbor search across millions or billions of vectors.

2. Retrieval-Augmented Generation (RAG)

RAG is a powerful technique that combines information retrieval with text generation. The process involves two main steps: 1. Retrieval: Given a user query, the system retrieves relevant context from a knowledge base using semantic search over vector embeddings. 2. Generation: The retrieved context is then provided to a large language model (LLM) along with the original query to generate an informed response. This approach allows LLMs to access up-to-date information without requiring retraining, making it ideal for question-answering systems, chatbots, and knowledge management applications.

3. Understanding Embeddings

Embeddings are dense vector representations of data that capture semantic meaning. In natural language processing, sentence embeddings transform text into fixed-size vectors where similar meanings result in vectors that are close together in the embedding space. Popular embedding models include: - OpenAI's text-embedding-3-large (3072 dimensions) - text-embedding-3-small (1536 dimensions) - Sentence-BERT variants The dimensionality-curse challenge: As dimension increases, the volume of space increases exponentially, making traditional indexing methods inefficient.

4. ChromaDB Overview

ChromaDB is an open-source embedding database designed for AI applications. Key features include:

- Simple API for adding, querying, and managing embeddings
 - Persistent storage with SQLite backend
 - Built-in support for metadata filtering
 - Compatibility with various embedding models
 - In-memory and persistent modes
- ChromaDB uses HNSW algorithm for efficient similarity search and supports multiple distance metrics including cosine similarity, L2 distance, and inner product.

5. Best Practices for RAG Systems

When building production RAG systems, consider these best practices: 1. **Chunk Size:** Balance between too small (loses context) and too large (dilutes relevance). Typical range: 500-1500 tokens. 2. **Metadata:** Store rich metadata (timestamps, sources, sections) to enable filtering and debugging. 3. **Hybrid Search:** Combine vector search with keyword-based search for better precision. 4. **Reranking:** Use a cross-encoder model to rerank top-k results for improved relevance. 5. **Monitoring:** Track retrieval quality metrics and user feedback to improve the system over time.